

Network Latency in Distributed LLM Inference

PROGRESS UPDATE: MARCH 25, 2026

Kelsey Penners

Quick Progress Update

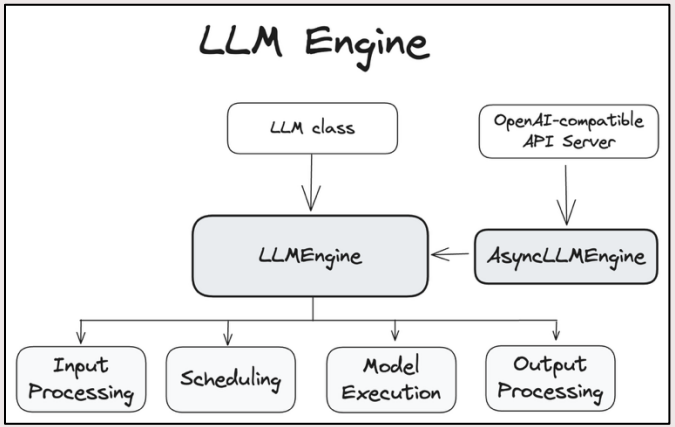
Writing:

- Background chapter outlined / partly drafted (LLM inference, parallelism strategies, metric definitions, vllm internals, etc)
- Single-GPU methodology section drafted, results section started

Other:

- Researched open questions from last few meetings (vllm scheduler, prefix caching, dataset justification)
- Phase 2 design drafted, scripts started

vLLM Scheduler



Source: vllm docs https://docs.vllm.ai/en/stable/design/arch_overview/

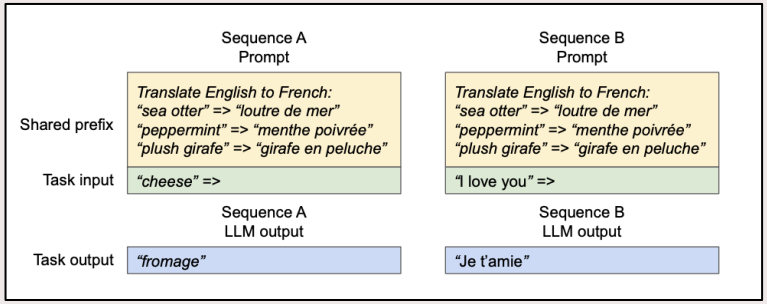
- Engine iterates through requests each step: schedule request, forward pass, process outputs
- Optimizations:
  - Continuous batching: adds requests as soon as slot opens
  - Chunked prefill: larger prefills are split into chunks, done across multiple steps

Why does inter-token latency rise with concurrency?

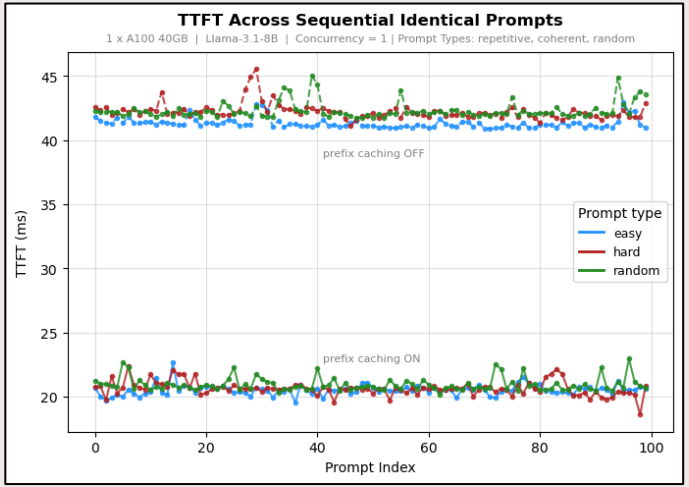
- At *n* concurrent requests, each decode step processes *n* sequences: more KV lookups, longer decode step

Prefix Caching

- Each KV cache block (16 tokens) gets a hash computed by token IDs before and within current block
- If a new request produces hashes of existing KV blocks, they're reused, prefill computation is skipped
- TTFT is reduced for repeated / similar prompts, does not affect decode



source: vllm paper (https://arxiv.org/pdf/2309.06180)



Datasets

Dataset	Typical ISL / OSL
ShareGPT	variable (conversational)
ArXiv Summarization	long (papers) / short (abstracts)
LongBench	long (up to 32k) / variable
Alpaca	short / short
Synthetic / Random	controlled

From papers like vLLM Paged Attention, Etalon, Vidur, LLM-Inference-Bench

- Depends on what workloads should be represented: translation, summarization, generation, conversation
- Synthetic or random tested in a paper discussing best practices for system-level inference performance benchmarking
  - Conclude that synthetic accurately represents real workload characteristics, max 7% difference in throughput across sequence lengths from 1024 – 16384 across various concurrencies

MLSys

acm Association for Computing Machinery

TU/e

Plan for Multi-GPU

Models: Llama 3.1-8B (baseline) & 70B (requires more GPUs)

- Maybe Mixtral-8x7B is a sparse model that is comparable to 70B model

Servers: funnotch (PCIe) | shyfec (NVLink, then NVLink disabled)

TP configurations: 2, 4, 5, 8 on **shy-fec** | 2, 4 on **funnotch**

Workload:

Concurrency sweep continued with fixed ISL/OSL or ShareGPT dataset?

Latency injection:

Norman's tool? If it works on server (Docker issue) and gives me control of the variables I want

Expected behavior:

ITL will increase, but TTFT effect should be minimal. KV cache saturation point will shift as memory is distributed

Next Steps and Discussion Points

Timeline:

- Draft up single-GPU results, phase 2 planning and scripting
- Multi-GPU baseline (no injection, TP sweep)
- Following weeks: latency injection, A100 NVLink vs PCIe comparison
- Research and writing in parallel!

Discussion:

- Project plan and thesis stuff
- Holiday?
- Latency injection: Norman's vllm-probe or NCCL call wrapping?

Thank you!

KV Cache Saturation

required KV cache size = 2 * batch size * context * precision * num layers * (num heads*head dim)

reserved KV cache mem = (Total GPU mem * gpu_memory_utilization)
- Model weights - Peak activations
- Framework overhead

